

Raksha

Enterprise AI Security & Governance Platform

Document: Comprehensive Technical Reference, Architecture Blueprint, Port Specification, Secrets Generation Handbook & Docker Deployment Guide

Document Code: RAKSHA-DOC-TECH-2026-V2.4

Classification: Enterprise Confidential - Internal Engineering & DevOps

Version: 2.4.0 (Enterprise Production Baseline)

Publication Date: October 2026

Audience: Platform Architects, Security Engineers, DevOps / SRE, Infrastructure Administrators

Executive Summary

Raksha is an enterprise-grade Generative AI security, governance, and observability platform. It safeguards enterprise assets through a dual-plane architecture:

- 1. Inference Gateway & API Plane:** A high-throughput reverse proxy (powered by Go 1.26+ **fasthttp**) mediating API traffic between internal applications and external LLM providers (e.g., OpenAI, Anthropic, Bedrock, Ollama). It enforces RBAC, virtual key budgeting, prompt guardrails, semantic caching, and full audit logging.
- 2. Endpoint & Browser Guard Plane:** A fleet of intelligent workstation agents and local MitM proxies (powered by Python 3.11+ and **mitmproxy**) deployed across corporate laptops. It intercepts browser sessions to target AI websites (e.g., ChatGPT, Claude, Gemini), enforcing Data Loss Prevention (DLP), PII redaction, sensitive file upload blocking, and user compliance warning banners.

```
+-----+ | RAKSHA ENTERPRISE
PLATFORM SUMMARY | +-----+
System Title | Raksha Enterprise Generative AI Security Platform | | Backend Engine | Go 1.26.4 (fasthttp,
GORM, CGO SQLite3 static, automaxprocs)| | Frontend Dashboard | React 19.2.3, TypeScript 5.9.2, Vite 8.0.16,
Tailwind 4.1 | | Endpoint Agent | Python 3.11+, mitmproxy 10+, PyInstaller, Inno Setup 6 | | Container Base |
Alpine Linux 3.23.4 (minimal hardened runtime container) | | Primary Database | PostgreSQL 16 (SCRAM-SHA-256
TLS) | | AI Inference Engine | Ollama (Local LLMs) & ~28 Cloud Provider SDKs | | Ingress / Edge Proxy | Nginx
/ Caddy / Cloudflare (Ports 80 / 443) |
+-----+
```

Security & Compliance Notice:

This document contains proprietary infrastructure specifications, port mappings, and cryptographic procedures. Distribution is strictly restricted to authorized platform administrators and DevOps personnel.

Table of Contents

Section	Topic	Primary Subject Matter
Section 1	System Architecture & Dual-Plane Topology	Core Architecture, Data Plane vs Control Plane, Request Lifecycle
Section 2	Technology Stack & Exact Version Matrix	Go 1.26, React 19, TypeScript 5.9, Python 3.11+, Alpine, PostgreSQL
Section 3	Network Topology & Comprehensive Port Matrix	Ports 6000, 3100, 18182, 18103, 18195, 18183, 5432, 11434, 80, 443
Section 4	Docker Download, Installation & Command Handbook	Docker Engine & Compose installation (Ubuntu/Debian/CentOS/Windows) & complete CLI handbook
Section 5	Cryptographic Secrets & Certificate Generation	OpenSSL generation for Guard Secret, Encryption Key, Password Reset, CA Root, Certbot
Section 6	Desktop Browser Guard Fleet Architecture	Laptop MitM agent, PAC distribution, OS trust store deployment (certutil, security)
Section 7	Production Environment Configuration (.env)	Complete annotated .env specification and security variable matrix
Section 8	Health Checks, Diagnostics & Operational Verification	Verification commands, container healthchecks, and troubleshooting matrix

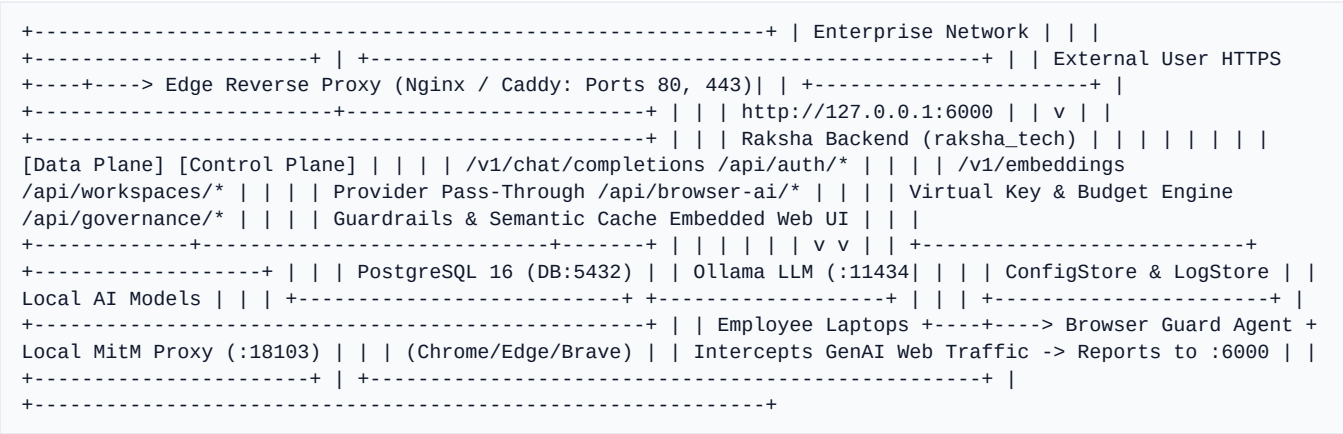
Document Conventions

- **CLI Commands:** All terminal commands are presented in copy-paste ready blocks.
- **Paths:** Linux/Docker paths use forward slashes (/app/data); Windows paths use backslashes (C:\Program Files\Raksha).
- **Secrets:** Placeholder tokens such as **change-me-long-random-secret** must be replaced using the cryptographic generation commands in **Section 5**.
- **Variables:** Environment variables referenced throughout this document correspond directly to the root .env configuration file.

1. System Architecture & Dual-Plane Topology

Raksha is designed around a dual-plane architecture housed within a unified deployment footprint. The platform separates high-performance data processing from governance and endpoint management.

1.1 Architecture Topology Diagram



1.2 Data Plane vs Control Plane

1. The Data Plane (**/v1/***, **Provider Pass-Throughs**):
- High-performance, low-latency Go runtime (**valyala/fasthttp**).
 - Translates incoming OpenAI-compatible requests to ~28 downstream AI providers (OpenAI, Anthropic, Bedrock, Gemini, Cohere, Ollama, etc.).
 - Applies pre-flight token bucket rate limits, budget checks, semantic cache lookups, and PII/prompt injection guardrails.
 - Logs request metadata asynchronously to the PostgreSQL **logstore** partition without blocking client streaming.
2. The Control Plane (**/api/*** and **Embedded Dashboard**):
- Serves the React 19 enterprise administration dashboard.
 - Manages Users, Role-Based Access Control (RBAC with 33 resources and 6 operations), Workspaces, and SCIM directory sync.
 - Governs Virtual Keys, Model Routing tables, Circuit Breakers, and Provider Credentials.
 - Orchestrates the **Browser AI** control plane: Target Websites, Guard Rules (Block, Redact, Warn), Fleet Agent configurations, and tamper-resistant uninstall keys.

2. Technology Stack & Exact Version Matrix

The table below outlines every language, runtime, library, and framework utilized in Raksha, along with the pinned version and repository source of truth.

2.1 Complete Language & Component Matrix

Layer / Component	Technology	Exact Version	Source of Truth File
Backend Core	Go (Golang)	1.26.4	go.work, core/go.mod, transports/raksha-http/go.mod

Layer / Component	Technology	Exact Version	Source of Truth File
HTTP Server Engine	valyala/fasthttp	v1.69.0	transports/raksha-http/go.mod
HTTP Routing	fasthttp/router	v1.5.4	transports/raksha-http/go.mod
Database ORM	GORM	v1.31.1	framework/configstore/go.mod
Embedded DB Driver	mattn/go-sqlite3	v1.14.33 (CGO static)	transports/raksha-http/go.mod, deploy/docker/Dockerfile.local
CPU Tuning Engine	uber-go/automaxprocs	v1.6.0	transports/raksha-http/main.go
Frontend Framework	React	19.2.3	ui/package.json
Frontend Language	TypeScript	5.9.2	ui/package.json
UI Build System	Vite	8.0.16	ui/package.json
Styling Engine	Tailwind CSS	4.1.12	ui/package.json
UI Navigation	TanStack React Router	1.168.10	ui/package.json
UI Data Tables	TanStack React Table	8.21.3	ui/package.json
State Management	Redux Toolkit	2.8.2	ui/package.json
UI Primitives	Radix UI	Latest Stable (1.1-1.3)	ui/package.json
Code Editor	Monaco Editor	0.52.2	ui/package.json
Data Visualization	Recharts	3.8.1	ui/package.json
Schema Validation	Zod	4.2.1	ui/package.json
Node Build Runtime	Node.js Alpine	Node 25-alpine	deploy/docker/Dockerfile.local (Stage 1)
Node Host Minimum	Node.js LTS	Node 20.x or 22.x LTS	ui/package.json (@types/node: 20.19.39)
Linter & Formatter	Oxlint / Oxfmt	1.60.0 / 0.45.0	ui/package.json
Desktop Guard Agent	Python	3.11+ / 3.12	apps/browser-guard/requirements-guard.txt
Interception Engine	mitmproxy	>= 10.0.0	apps/browser-guard/requirements-guard.txt
Binary Compiler	PyInstaller	>= 6.0.0	apps/browser-guard/requirements-guard.txt
Image Processing	Pillow	>= 10.0.0	apps/browser-guard/requirements-guard.txt
PDF Inspection Engine	pypdf	>= 4.0.0	apps/browser-guard/requirements-guard.txt
Windows Installer	Inno Setup	6.2+	apps/browser-guard/installer/
macOS Packager	Apple pkgbuild	macOS Native	apps/browser-guard/Makefile

Layer / Component	Technology	Exact Version	Source of Truth File
Runtime Container OS	Alpine Linux	3.23.4	deploy/docker/Dockerfile.local (Stage 3)
Enterprise Database	PostgreSQL	16.x	docker-compose.yml, .env.example
Local LLM Engine	Ollama	v0.3.x / v0.5.x	docker-compose.yml (ollama service)

3. Network Topology & Comprehensive Port Matrix

Understanding the required network ports is essential for setting up firewalls (UFW, AWS Security Groups, Azure NSGs) and configuring container bindings.

3.1 Comprehensive Port Allocation Table

Port Number	Protocol	Default Config Key	Scope / Binding	Service Name	Function & Purpose
6000	TCP	APP_PORT	0.0.0.0:6000 (Docker)	raksha_tech Backend	Main Go HTTP server. Handles UI dashboard, /v1/* inference proxy, /api/* management plane, and Browser Guard fleet reporting.
3100	TCP	UI_PORT	127.0.0.1:3100	UI Vite Dev Server	Used exclusively in local development (npm run dev in ui/). In production, UI is statically compiled into the Go binary on port 6000.
18182	TCP	PROXY_PORT	0.0.0.0:18182	raksha_browser_a i_proxy	Shared Docker network proxy container (mitmproxy). Optional profile for lab/office PAC setups where endpoints route to a centralized proxy.
18103	TCP	RAKSHA_PROXY_ADD R	127.0.0.1:18103 (Local)	Guard Laptop MitM Proxy	Runs locally on each employee workstation. Browser sends traffic through this local port; the agent inspects requests before forwarding them to AI sites.

Port Number	Protocol	Default Config Key	Scope / Binding	Service Name	Function & Purpose
18195	TCP	PAC_HTTP_PORT	127.0.0.1:18195 (Local)	Guard Local PAC Server	Runs locally on employee laptops to serve the dynamic PAC (Proxy Auto-Config) script to Windows/macOS network settings. Falls back to +1, +2 if occupied.
18183	TCP	MITM_WEB_PORT	127.0.0.1:18183 (Local)	Dev MitM Web UI	Diagnostic web interface for mitmproxy during local agent debugging (start_app.bat / start_app.sh). Never exposed in production.
5432	TCP	DB_PORT	Internal / Private VPC	PostgreSQL Database	Primary enterprise database storing configstore (tenants, users, keys, rules) and logstore (inference logs, DLP audit logs).
11434	TCP	OLLAMA_URL	Docker Bridge / Loopback	Ollama AI Service	Local LLM inference engine. Exposes REST API for llama3, mistral, deepseek, and nomic-embed-text models.
80	TCP	System Port	0.0.0.0:80 (Host Public)	Nginx / Reverse Proxy	Inbound plain HTTP. Used for Let's Encrypt Certbot ACME domain verification and automatic 301 redirection to HTTPS (port 443).
443	TCP	System Port	0.0.0.0:443 (Host Public)	Nginx / Reverse Proxy	Inbound secure HTTPS. Edge entry point terminating TLS certificates, offloading SSL, and reverse proxying to http://127.0.0.1:6000 .
6379	TCP	REDIS_PORT	Internal (Optional)	Redis In-Memory Cache	Optional Redis cache used for semantic cache embedding indexing and distributed rate limiting across multi-node clusters.

Port Number	Protocol	Default Config Key	Scope / Binding	Service Name	Function & Purpose
6333 / 6334	TCP	QDRANT_PORT	Internal (Optional)	Qdrant Vector DB	Optional vector storage engine (HTTP: 6333, gRPC: 6334) for semantic prompt matching and enterprise RAG.
9000	TCP	WEAVIATE_PORT	Internal (Optional)	Weaviate Vector Store	Optional vector database engine for high-scale enterprise embedding storage.

3.2 Network Perimeter & Firewall Security Matrix

+-----+ NETWORK FIREWALL SECURITY RULES +-----+-----+-----+-----+-----+-----+					
Rule Type	Port	Allowed Source	Policy & Security Rationale		
+-----+-----+-----+-----+-----+-----+ PUBLIC IN 443					
0.0.0.0/0 (All) Main HTTPS endpoint for UI and APIs PUBLIC IN 80 0.0.0.0/0 (All) HTTP to HTTPS					
redirect & ACME Certbot INTERNAL IN 6000 127.0.0.1 / Proxy Protect Go backend from direct access					
INTERNAL IN 5432 App Containers / VPC Strictly protect DB from public web INTERNAL IN 11434					
App Containers / VPC Restrict LLM API to internal backend ENDPOINT IN 18103 127.0.0.1 (Loopback)					
Employee proxy bound only to localhost ENDPOINT IN 18195 127.0.0.1 (Loopback) Employee PAC server					
bound to localhost +-----+-----+-----+-----+-----+-----+					

4. Docker Download, Installation & Command Handbook

Raksha is designed to run seamlessly via Docker and Docker Compose. This section provides complete installation commands across all operating systems and an exhaustive operations handbook.

4.1 Installing Docker Engine & Docker Compose

Method A: Ubuntu Linux (20.04 / 22.04 / 24.04 LTS) and Debian (11 / 12)

Execute with **sudo** permissions:

```
# 1. Update existing packages sudo apt update && sudo apt upgrade -y # 2. Install prerequisite packages sudo
apt install -y ca-certificates curl gnupg lsb-release # 3. Add Docker official GPG key sudo install -m 0755
-d /etc/apt/keyrings curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg sudo chmod a+r /etc/apt/keyrings/docker.gpg # 4. Set up the Docker repository
echo \ "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \ $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \ sudo
tee /etc/apt/sources.list.d/docker.list > /dev/null # 5. Install Docker Engine, CLI, containerd, and Docker
Compose plugin sudo apt update sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin # 6. Enable and start Docker system service sudo systemctl enable docker sudo systemctl
start docker # 7. (Optional) Allow current non-root user to run Docker commands sudo usermod -aG docker $USER
```

Automated One-Liner Alternative for Linux Servers:

```
curl -fsSL https://get.docker.com -o get-docker.sh && sudo sh get-docker.sh
```

Method B: CentOS / RHEL (8 / 9) / Rocky Linux

```
# 1. Remove old versions sudo dnf remove -y docker docker-client docker-latest docker-common # 2. Add Docker repository sudo dnf install -y dnf-plugins-core sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo # 3. Install Docker Engine and Compose sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin # 4. Start and enable service sudo systemctl enable --now docker
```

Method C: Microsoft Windows 10 / 11 (Development Workstations)

1. Install WSL2 (Windows Subsystem for Linux):

```
wsl --install
```

2. Download Docker Desktop for Windows from the official portal:

<https://docs.docker.com/desktop/install/windows-install/>

3. During installation, ensure the checkbox **"Use WSL 2 instead of Hyper-V"** is enabled.
4. Verify Docker in PowerShell:

```
docker --version docker compose version
```

4.2 Network Prerequisites & Setup

Raksha requires two Docker bridge networks:

1. **raksha-network**: Primary internal bridge network linking the backend container with auxiliary services.
2. **1panel-network**: Dedicated network linking the local Ollama LLM container to the backend.

```
# Create the external Ollama network (must exist before running compose up) docker network create 1panel-network # Create the primary application bridge network (if not auto-created) docker network create raksha-network # Verify that both networks are active docker network ls
```

4.3 Building & Compiling Raksha Docker Images

The backend image (**deploy/docker/Dockerfile.local**) utilizes a multi-stage Docker build:

- **Stage 1 (UI)**: **node:25-alpine** executes **npm run build:raksha** to produce the optimized React Vite bundle.
- **Stage 2 (Go)**: **golang:1.26.4-alpine** links SQLite3 statically and compiles the Go binary **/app/main**.
- **Stage 3 (Runtime)**: **alpine:3.23.4** packages the binary, entrypoint script, and Guard proxy files with minimal overhead.

```
# 1. Standard build from project root docker compose build # 2. Build without Docker cache (forces complete clean recompilation of UI & Go) docker compose build --no-cache # 3. Build and launch in a single command docker compose up -d --build
```

4.4 Container Startup & Multi-Profile Orchestration

```
# 1. Start primary Raksha backend container in detached mode (background) docker compose up -d # 2. Start primary backend AND the optional shared network proxy docker compose --profile network-proxy up -d # 3. Start in foreground mode (prints real-time logs to console; Ctrl+C to halt) docker compose up
```

4.5 Container Inspection, Logs & Real-Time Monitoring

```
# 1. View status of all running containers, port bindings, and health checks docker compose ps # 2. View all containers (including stopped or exited containers) docker compose ps -a # 3. Stream live logs from all running services docker compose logs -f # 4. Stream live logs from the backend container with the last 100 lines docker compose logs -f --tail=100 raksha_tech # 5. Stream logs from the network proxy container docker compose logs -f --tail=100 raksha_browser_ai_proxy # 6. Filter logs for errors or warnings docker compose logs raksha_tech | grep -i "error" # 7. Check container resource utilization (CPU %, RAM, Network I/O, PIDs) docker stats raksha_tech # 8. Inspect detailed low-level JSON configuration (IP address, mounts, environment) docker inspect raksha_tech
```

4.6 In-Container Execution & Interactive Debugging

```
# 1. Open an interactive shell inside the running Raksha container docker exec -it raksha_tech /bin/sh # 2. Test the internal healthcheck endpoint from inside the container docker exec -it raksha_tech wget -qO- http://127.0.0.1:6000/health # 3. Verify environment variables loaded into the container process docker exec -it raksha_tech env | grep -E "APP_PORT|DB_HOST|SERVER_DOMAIN" # 4. Verify volume mounts and permissions on data directory docker exec -it raksha_tech ls -la /app/data
```

4.7 Graceful Shutdown, Teardown & Resource Pruning

```
# 1. Gracefully restart the backend container (e.g., after modifying .env) docker compose restart raksha_tech # 2. Stop running containers without removing container instances or networks docker compose stop # 3. Stop and tear down containers, networks, and internal links docker compose down # 4. Stop and remove containers along with internal anonymous volumes (data bind mounts preserved) docker compose down -v # 5. Clean up stopped containers, unused networks, and dangling images docker system prune -f # 6. Nuclear cleanup: remove all unused images, stopped containers, and volumes (CAUTION) docker system prune -a --volumes -f
```

5. Cryptographic Secrets, Tokens & SSL/TLS Certificate Generation

Raksha enforces strict, zero-trust cryptographic requirements. Secrets must never use default values in production. All commands below generate cryptographically secure, production-grade keys.

5.1 Summary of Platform Secrets

Secret Environment Variable	Recommended Key Length	Algorithm / Purpose
RAKSHA_GUARD_SECRET	256-bit (32 bytes Base64)	Fleet endpoint mutual authentication token
RAKSHA_ENCRYPTION_KEY	256-bit (32 bytes Base64)	AES-256-GCM encryption at rest for DB credentials
PASSWORD_RESET_SECRET	384-bit (48 bytes Base64)	HMAC secret for password recovery tokens (min 32 chars)
RAKSHA_METRICS_TOKEN	256-bit (32 bytes Hex)	Bearer token for external Prometheus scraping
CLUSTER_REPLICATE_SECRET	384-bit (48 bytes Base64)	Shared HMAC secret for multi-node cluster KV replication

5.2 Secret Generation CLI Commands

Execute these commands in your server terminal to generate values for `.env`:

```
# 1. Generate RAKSHA_GUARD_SECRET (Mutual authentication between backend and desktop agents) openssl rand
-base64 32 # Example output: vR8sQ9k4N2p1Xy7Z8b0A3c5D6e7F8g9H0i1J2k3L4m5= # 2. Generate
RAKSHA_ENCRYPTION_KEY (AES encryption at rest for provider keys in PostgreSQL) openssl rand -base64 32 #
Example output: aB3dE5gH7iJ9kL1mN3oP5qR7sT9uV1wX3yZ5aB7cD9e= # 3. Generate PASSWORD_RESET_SECRET
(Cryptographic HMAC for user password resets; min 32 chars) openssl rand -base64 48 # Example output:
K9jL2mN5pQ8rT1vW4xY7zA0bC3dE6fG9hI2jK5lM8nO1pQ4rS7tU0vW3xY6zA9bC= # 4. Generate RAKSHA_METRICS_TOKEN
(Prometheus scraping bearer token) openssl rand -hex 32 # Example output:
e4c9a18b7f205391d836ae05c317b9021849f12d5918a3627d048bce91a27e01 # 5. Generate CLUSTER_REPLICATE_SECRET
(Multi-node KV replication secret) openssl rand -base64 48 # Example output:
9f8a7b6c5d4e3f2a1b0c9d8e7f6a5b4c3d2e1f0a9b8c7d6e5f4a3b2c1d0e9f8a=
```

Automated Command to Populate **.env** with Fresh Secrets:

```
# Run this one-liner on a newly created .env file to generate and insert all secrets automatically: sed -i
"s|^RAKSHA_GUARD_SECRET=.*|RAKSHA_GUARD_SECRET=$(openssl rand -base64 32)|" .env sed -i
"s|^RAKSHA_ENCRYPTION_KEY=.*|RAKSHA_ENCRYPTION_KEY=$(openssl rand -base64 32)|" .env sed -i
"s|^PASSWORD_RESET_SECRET=.*|PASSWORD_RESET_SECRET=$(openssl rand -base64 48)|" .env sed -i
"s|^RAKSHA_METRICS_TOKEN=.*|RAKSHA_METRICS_TOKEN=$(openssl rand -hex 32)|" .env sed -i
"s|^CLUSTER_REPLICATE_SECRET=.*|CLUSTER_REPLICATE_SECRET=$(openssl rand -base64 48)|" .env
```

5.3 Public SSL/TLS Certificate Acquisition via Let's Encrypt (Certbot)

For production internet-facing servers with a public domain (e.g., **raksha.yourcompany.com**):

```
# 1. Install Certbot sudo apt update && sudo apt install -y certbot python3-certbot-nginx # 2. Standalone
Mode (run before starting Nginx or while port 80 is free) sudo certbot certonly --standalone \
--preferred-challenges http \ -d raksha.yourcompany.com \ --email secops@yourcompany.com \ --agree-tos \
--no-eff-email # Certificates are saved to: # /etc/letsencrypt/live/raksha.yourcompany.com/fullchain.pem #
/etc/letsencrypt/live/raksha.yourcompany.com/privkey.pem # 3. Configure Automated Certificate Renewal (runs
twice daily via crontab) echo "0 3,15 * * * root certbot renew --quiet --post-hook 'systemctl reload nginx'"
| sudo tee -a /etc/crontab # 4. Dry-run test of renewal mechanism sudo certbot renew --dry-run
```

5.4 Private Self-Signed TLS Certificate Generation with OpenSSL

For internal enterprise VPCs, air-gapped labs, or private testing:

```
# 1. Create certificate directory sudo mkdir -p /etc/ssl/raksha && cd /etc/ssl/raksha # 2. Generate 4096-bit
RSA key and self-signed certificate with SAN valid for 3 years (1095 days) sudo openssl req -x509 -nodes
-days 1095 -newkey rsa:4096 \ -keyout raksha_server.key \ -out raksha_server.crt \ -subj
"/C=US/ST=California/L=SanFrancisco/O=YourCompany/OU=IT/CN=raksha.yourcompany.com" \ -addext
"subjectAltName=DNS:raksha.yourcompany.com,DNS:localhost,IP:127.0.0.1" # 3. Set secure file permissions sudo
chmod 600 raksha_server.key sudo chmod 644 raksha_server.crt # 4. Verify certificate modulus matches private
key openssl x509 -noout -modulus -in raksha_server.crt | openssl md5 openssl rsa -noout -modulus -in
raksha_server.key | openssl md5 # The two output MD5 hashes MUST match identically!
```

5.5 Browser Guard Root CA Generation for Interception Proxy

Browser Guard intercepts HTTPS connections between employee browsers and target Generative AI platforms (such as OpenAI and Anthropic) to inspect prompts and attachments. This requires an internal **Root Certificate Authority (CA)**.

```
# 1. Create directory for Root CA mkdir -p /opt/raksha/certs/ca && cd /opt/raksha/certs/ca # 2. Generate Root
CA Private Key (4096-bit RSA) openssl genrsa -out raksha-ca.key 4096 chmod 400 raksha-ca.key # 3. Generate
Root CA Certificate (valid for 10 years / 3650 days) openssl req -x509 -new -nodes -key raksha-ca.key -sha256
-days 3650 \ -out raksha-ca.crt \ -subj "/C=US/ST=State/L=City/O=Raksha Enterprise Security/OU=Fleet
Security/CN=Raksha Root CA - Enterprise AI Guard" # 4. Create combined PEM file for mitmproxy cat
raksha-ca.key raksha-ca.crt > mitmproxy-ca.pem chmod 600 mitmproxy-ca.pem
```

5.6 Deploying the Root CA to Client OS Trust Stores

To prevent browser security warnings (**ERR_CERT_AUTHORITY_INVALID**), deploy **raksha-ca.crt** to endpoint trust stores.

A. Windows Client Installation (Automated via CMD / PowerShell / Inno Setup)

Execute as **Administrator**:

```
certutil -addstore -f "Root" "C:\path\to\raksha-ca.crt"
```

Verify installation:

```
certutil -verifystore "Root" "Raksha Root CA - Enterprise AI Guard"
```

B. macOS Client Installation (Terminal / Jamf / MDM)

Execute with **sudo**:

```
sudo security add-trusted-cert \ -d \ -r trustRoot \ -k /Library/Keychains/System.keychain \
/path/to/raksha-ca.crt
```

C. Linux (Ubuntu / Debian) Workstation Installation

```
sudo cp raksha-ca.crt /usr/local/share/ca-certificates/raksha-ca.crt sudo update-ca-certificates
```

D. Mozilla Firefox NSS Database Installation

Firefox maintains an independent certificate store:

```
for certDB in $(find $HOME/.mozilla/firefox* -name "cert9.db"); do certdir=$(dirname ${certDB}) certutil -A
-n "Raksha Root CA" -t "TCu,Cu,Tu" -i /path/to/raksha-ca.crt -d sql:${certdir} done
```

6. Desktop Browser Guard Fleet Architecture & Deployment

The Browser Guard desktop agent (**apps/browser-guard**) runs silently on employee laptops, providing zero-friction data protection without breaking the user experience.

6.1 Interception Workflow

```
+-----+ | BROWSER GUARD
INTERCEPTION PIPELINE |
+-----+ | 1. Employee
navigates to https://chatgpt.com in Google Chrome / Microsoft Edge | | 2. Workstation routing directs
connection through local proxy (127.0.0.1:18103) | | 3. MitM Proxy inspects HTTP POST request body & attached
files | | 4. Policy Engine checks against Guard Rules fetched from backend (:6000) | | - PII Detected? ->
Automatically redacted before reaching AI provider | | - Source Code / Financial Data? -> Request blocked
with HTML security modal | | - General AI Prompt? -> Allowed, metadata logged to Postgres LogStore | | 5.
MitM Proxy forwards sanitized request to chatgpt.com over upstream TLS |
+-----+
```

6.2 Key Operational Parameters

- **RAKSHA_PROXY_ADDR (127.0.0.1:18103)**: The local bind address for the MitM proxy.
- **PAC_HTTP_PORT (127.0.0.1:18195)**: Serves the Proxy Auto-Configuration (PAC) script to the OS network settings.
- **RAKSHA_GUARD_REQUIRE_SECRET=1**: Fail-closed mode. Rejects any agent communication if **RAKSHA_GUARD_SECRET** does not match.

7. Production Environment Configuration (.env)

Below is the standard production **.env** reference file. Copy from **.env.example** and populate with generated secrets.

```
# ===== # RAKSHA ENTERPRISE
PRODUCTION CONFIGURATION (.env) #
===== # 1. Server & Public Domain
SERVER_DOMAIN=https://raksha.yourcompany.com APP_PORT=6000 APP_HOST=0.0.0.0 CONTAINER_NAME=raksha_tech
DOCKER_NETWORK=raksha-network # 2. Proxy Ports PROXY_PORT=18182
NETWORK_PROXY_CONTAINER_NAME=raksha_browser_ai_proxy RAKSHA_PROXY_ADDR=127.0.0.1:18103 PAC_HTTP_PORT=18195
MITM_WEB_PORT=18183 # 3. UI Dashboard Settings UI_PORT=3100 RAKSHA_BACKEND_URL=http://localhost:6000 # 4.
PostgreSQL Enterprise Database DB_TYPE=postgres DB_HOST=10.0.1.50 DB_PORT=5432 DB_NAME=raksha_prod
DB_USER=raksha_admin DB_PASSWORD=StrongDatabasePasswordGeneratedWithOpenSSL! DB_SSL_MODE=require # 5. Local
Ollama LLM Service OLLAMA_URL=http://host.docker.internal:11434 OLLAMA_DOCKER_NETWORK=1panel-network # 6.
Initial Administrator Credentials (change after first login) ADMIN_EMAIL=security-admin@yourcompany.com
ADMIN_PASSWORD=InitialStrongPasswordToChange! # 7. Logging & Persistence LOG_LEVEL=info LOG_STYLE=json
APP_DIR=/app/data # 8. Branding RAKSHA_COMPANY_NAME=Enterprise AI Security Group
RAKSHA_COMPANY_LOGO=/company-logo.png # 9. Cryptographic Keys & Secrets (Generated in Section 5)
RAKSHA_ENCRYPTION_KEY=vR8sQ9k4N2p1Xy7Z8b0A3c5D6e7F8g9H0i1J2k3L4m5=
RAKSHA_GUARD_SECRET=aB3dE5gH7iJ9kL1mN3oP5qR7sT9uV1wX3yZ5aB7cD9e= RAKSHA_GUARD_REQUIRE_SECRET=1
RAKSHA_PAC_ALLOW_QUERY_PROXY=0
RAKSHA_METRICS_TOKEN=e4c9a18b7f205391d836ae05c317b9021849f12d5918a3627d048bce91a27e01
PASSWORD_RESET_SECRET=K9jL2mN5pQ8rT1vW4xY7zA0bC3dE6fG9hI2jK5lM8n01pQ4rS7tU0vW3xY6zA9bC=
CLUSTER_REPLICATE_SECRET=9f8a7b6c5d4e3f2a1b0c9d8e7f6a5b4c3d2e1f0a9b8c7d6e5f4a3b2c1d0e9f8a= # 10. Security &
Edge Proxies TRUST_PROXY_HEADERS=1 TRUSTED_PROXIES=127.0.0.1,10.0.0.0/8,172.16.0.0/12,192.168.0.0/16
ALLOW_LOCALHOST_CORS=0
```

8. Health Checks, Diagnostics & Operational Verification

8.1 API & Container Health Verification

Execute these commands to verify that all layers of Raksha are operating correctly:

```
# 1. Primary Backend Health Check (verifies DB connection and router) curl -i http://localhost:6000/health #
Expected Output: HTTP/1.1 200 OK {"status":"healthy"} # 2. Inspect Prometheus Metrics (requires Bearer token
if configured) curl -H "Authorization: Bearer <RAKSHA_METRICS_TOKEN>" http://localhost:6000/metrics # 3.
Verify Container Status via Docker Compose docker compose ps # 4. Check PostgreSQL Database Connection from
Host nc -zv 10.0.1.50 5432 # Expected: Connection to 10.0.1.50 port 5432 [tcp/postgresql] succeeded! # 5.
Check Ollama LLM Connection from Host curl -i http://127.0.0.1:11434/api/tags # Expected: HTTP/1.1 200 OK
with list of installed models
```

8.2 Operational Diagnostics & Resolution Matrix

Symptom / Error	Root Cause	Exact Resolution Command
failed to connect to PostgreSQL on port 5432	Database container not running or firewall blocking port	docker compose ps to check DB, verify credentials in .env , test with nc -zv DB_HOST 5432 .
network 1panel-network not found	External Ollama Docker network was not created prior to startup	Run docker network create 1panel-network , then rerun docker compose up -d .
Browser Guard: 401 Unauthorized on /api/*	Missing or mismatched RAKSHA_GUARD_SECRET between client & backend	Synchronize secret using python apps/browser-guard/scripts/sync_config_from_env.py .
Browser displays ERR_CERT_AUTHORITY_INVALID	Root CA certificate not installed in client operating system store	Run certutil -addstore -f "Root" raksha-ca.crt on Windows or security add-trusted-cert on macOS.
Port 6000 already in use / bind error	Another application or previous orphan container is holding port	Run lsof -i :6000 (or `netstat -ano`)
UI displays blank screen or 404 on assets	UI static build missing from Go bundle	Run npm run build:raksha inside ui/ , then rebuild container with docker compose up -d --build .

Technical Support & Document Revision

- Lead Repository: **d:\unifai_project**
- Configuration Root: **.env** & **configs/config.json**
- Release Tracking: Git Baseline **16d4c3e**
- Platform Maintenance: Raksha SecOps & Cloud Operations Team